

UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Elaborato finale in **Networks and Cloud Infrastructures**

Rilevamento e mitigazione di attacchi DoS in una rete SDN

Anno Accademico 2025/2026

Candidati
Massimo Marrone **matr. M63006784**
Carmine Bellotti **matr. M63001742**

Repository: `https://github.com/giovannistanco/NCIS_unina/tree/main/2026/Marrone_Bellotti`

Introduzione

Nelle reti *Software-Defined Networking* (SDN) il piano di controllo è separato dal piano dati: un controller centrale programma gli switch con regole OpenFlow e ne legge i contatori. Questa visione globale permette di difendere la rete da un attacco *Denial of Service* (DoS) in modo selettivo: un attaccante che inonda un collegamento condiviso può essere isolato senza penalizzare gli utenti onesti collegati allo stesso switch.

Il progetto realizza su una rete emulata un sistema che **monitora** il traffico, **rileva** gli attacchi e li **mitiga** bloccando solo il MAC dell'attaccante. Il punto di partenza è il baseline dell'anno precedente, di cui sono stati corretti sei difetti: over-blocking (blocco della porta intera, ora drop per MAC sorgente), logica tutta nel controller (ora blocklist condivisa, alimentabile anche dall'esterno), codice monolitico (ora moduli Monitor, Detector, Mitigator, Blocklist), soglia statica (ora soglia adattiva con tetti assoluti), solo DoS a rate costante (aggiunto un detector Machine Learning) e topologia banale (ora con un ciclo e due attaccanti su switch diversi). Ogni scelta è verificata con scenari Mininet ripetibili, con la soglia di successo fissata *prima* di ogni run.

1 Ambiente di emulazione e strumenti

- **Docker**: un container Debian 11 (Python 3.9, richiesto da Ryu) con tutte le dipendenze a versione fissata, avviato con `run.ps1` / `run.sh`;
- **Mininet** e **Open vSwitch**: host, switch OpenFlow 1.3 e collegamenti con banda e ritardo imposti (`TCLink`);
- **Ryu**: controller Python che estende lo switch L2 con Spanning Tree (`simple_switch_stp_13`) con i moduli di difesa;
- **iperf**, **hping3**: traffico legittimo e flood UDP, SYN, ICMP;
- **scikit-learn**: Random Forest e valutazione del detector ML.

2 Topologia e architettura

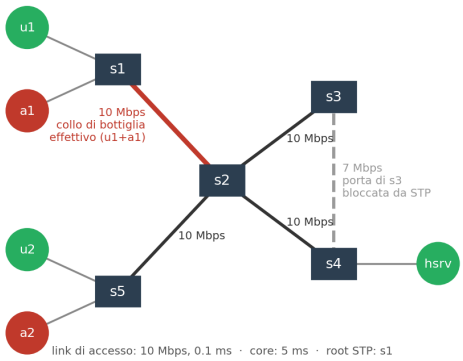


Figura 1: Topologia: 5 switch con ciclo `s2-s3-s4-s2`, server `hsrv`, utenti `u1`, `u2` e attaccanti `a1`, `a2`.

I collegamenti sono da 10 Mbps (`s3-s4` da 7 Mbps). Gli attaccanti `a1` (su `s1`) e `a2` (su `s5`) sono su switch diversi; l'utente legittimo `u1` è sullo **stesso switch** di `a1`. Dopo la convergenza (circa 35 s) STP spegne `s3-s4` e il traffico verso il server passa per `s1-s2-s4`: il **collo di bottiglia** è l'uplink `s1-s2`, condiviso da `u1` e `a1`.

Il controller aggiunge allo switch L2 quattro moduli disaccoppiati:

- **Monitor**: ogni 5 s legge le *flow stats* e calcola per ogni MAC sorgente pacchetti/s, byte/s, dimensione media del pacchetto e **traffico di ritorno** (`rx`, i pacchetti diretti verso di lei). Le regole di drop sono escluse.

- **Detector** (default: soglia adattiva): media e varianza EWMA su byte/s e pacchetti/s per sorgente, più due **tetti** che coprono il flood che parte già al massimo: 2000 pps sempre e 0,95 MB/s solo se la sorgente **non riceve risposte** (`rx` $<$ 10% di quanto manda). Un TCP legittimo riceve ACK (46–100%), un flood UDP no ($\leq$ 2%). Chi riceve più di quanto manda (il server sotto SYN flood) non viene mai segnalato. In alternativa `DETECTOR=ml` usa un Random Forest.
- **Conferma 2 su 3**: si blocca dopo 2 intervalli sospetti sugli ultimi 3.
- **Mitigator e Blocklist**: regola OpenFlow di **drop per MAC sorgente** a priorità 100 su tutti gli switch, con scadenza di 60 s; la whitelist è opzionale. OVS inoltra al controller al massimo 200 packet-in/s per switch, per proteggere il piano di controllo.

3 Funzionamento

Per riprodurre i risultati basta `run.sh` (o `run.ps1` su Windows), che apre una shell nel container, e poi `bash experiments/run_all.sh mit`, che esegue da solo la demo; il README del repository elenca tutti gli scenari e le opzioni. A mano, il sistema si esegue in tre terminali dentro il container: il controller (`ryu-manager controller.py`, con variabili come `WHITELIST=0` o `MITIGATE=0`), la rete (`python3 topology.py`) e un terminale di osservazione. Dopo la convergenza di STP, `pingall` conferma la raggiungibilità completa (0% persi, 20/20). `u1` avvia un download TCP verso `hsrv`; poi `a1` lancia un flood UDP a 25 Mbps, oltre il doppio della capacità dell'uplink:

```
mininet> a1 iperf -c 10.0.0.100 -u -p 5001 -b 25M -t 60 &
```

Per le figure il controller è avviato **senza whitelist** e con blocco permanente (`WHITELIST=0 BLOCK_TTL=0`).

```
Whitelist: OFF, BLOCK_TTL: permanente
Detector: EWMA, conferma 2/3, congestione: OFF

DETECT src=00:00:00:00:00:f1 bps=1337515 pps=885 pkt=1512B rx=0 (EWMA)
DETECT src=00:00:00:00:00:f1 bps=1234094 pps=816 pkt=1512B rx=0 (EWMA)
BLOCK 00:00:00:00:00:f1 (EWMA)
```

Figura 2: Configurazione del controller e rilevamento: due intervalli sospetti (`rx=0`: a1 non riceve risposte), poi il blocco del MAC di a1 (righe STP omesse).

Il controller misura per `a1` fino a 1,34 MB/s, cioè l'intero link, con `rx=0`: il primo `DETECT` arriva a +5,9 s dall'inizio del flood, il `BLOCK` a +10,9 s. In quell'intervallo il download di `u1` scende da 9,6 a 4,8 Mbps e poi quasi a zero; appena `a1` è bloccato torna a 9,56 Mbps e ci resta mentre l'attacco continua (il picco di 23 Mbps nel secondo successivo al blocco è la coda accumulata che si svuota).

```
mininet> u1 ping -c 3 hsrv
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.
64 bytes from 10.0.0.100: icmp_seq=1 ttl=64 time=1037 ms
64 bytes from 10.0.0.100: icmp_seq=2 ttl=64 time=1086 ms
64 bytes from 10.0.0.100: icmp_seq=3 ttl=64 time=1130 ms

--- 10.0.0.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2030ms
rtt min/avg/max/mdev = 1037.123/1084.591/1130.449/38.117 ms, pipe 2

mininet> a1 ping -c 3 hsrv
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.

--- 10.0.0.100 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2055ms
```

```
ovs-ofctl -O OpenFlow13 dump-flows s1 | grep priority=100
 cookie=0x0, duration=18.447s, table=0, n_packets=3258, n_bytes=4904046, priority=100,dl_src=00:00:00:00:00:f1 actions=d
rop
```

Figura 3: Dopo il `BLOCK`: `u1` raggiunge il server, `a1` perde il 100% dei ping. Sotto, la regola di drop su `s1`: priorità 100, `dl_src` di a1, 3258 pacchetti del flood scartati in 18 s.

I tempi di risposta di `u1` (circa 1 s) dipendono dal suo stesso download TCP, che riempie la coda del collegamento; prima dell'attacco, con lo stesso download in corso, erano già 0,6–0,8 s, mentre `a1` rispondeva in circa 46 ms.

La stessa scena è stata ripetuta in cinque configurazioni (Tabella 1). In tutte `u1` non è mai stato segnalato, e prima del flood scaricava a 9,5–9,6 Mbps.

Configurazione	Flood di `a1`	DETECT / BLOCK	`u1` (10 s / +20–40 s)	ping `a1`
Senza mitigazione (`MITIGATE=0`)	UDP 25 Mbps	+5,5 s / mai	2,4 / **1,0** Mbps	risponde
Con whitelist	UDP 25 Mbps	+4,6 / +9,6 s	1,6 / **9,6** Mbps	100% perso
Senza whitelist (`WHITELIST=0`)	UDP 25 Mbps	+5,4 / +10,4 s	1,4 / **9,6** Mbps	100% perso
ML (`DETECTOR=ml`)	UDP, pacchetti 250 B	+3,3 / +8,3 s	5,4 / **9,6** Mbps	100% perso
ML (`DETECTOR=ml`)	UDP 25 Mbps	mai / mai	2,0 / **0,5** Mbps	risponde

Tabella 1: Le cinque casistiche dal vivo: tempi dall'inizio del flood e throughput di `u1`.

Senza mitigazione il controller vede l'attacco a ogni intervallo ma non interviene, e `u1` resta a 1 Mbps: un DoS vero e proprio. La soglia adattiva blocca `a1` in circa 10 s con e senza whitelist. Il ML blocca il flood a pacchetti piccoli anche prima (+8,3 s), ma non rileva quello a pacchetti grandi, che ha le stesse feature di un UDP legittimo ad alta banda.

4 Risultati sperimentali

Negli scenari automatici (`experiments/run_all.sh`) `a1` manda 40 s di traffico normale e poi il flood per 60 s; in `cold` il flood parte da subito. Tempi misurati dall'inizio del flood; con polling a 5 s e conferma 2/3 il minimo è circa 10 s.

Scenario	Rilevamento e blocco	u1 segnalato
`mit` (whitelist attiva)	a1 bloccato a +9,8 s	no
`nowl` (senza whitelist)	a1 bloccato a +9,6 s	**no**
`both`: a1 + a2, senza whitelist, 8 run	entrambi bloccati a +8,8–10,4 s (8/8)	no
`cold`, EWMA del baseline	**mai rilevato**	–
`cold`, EWMA + tetti, 6 run	a1 bloccato a +12,2–12,5 s (6/6)	no
SYN flood verso il server, 4 run	a1 bloccato a +9,0–10,2 s; server mai bloccato	no

Tabella 2: Mitigazione negli scenari automatici.

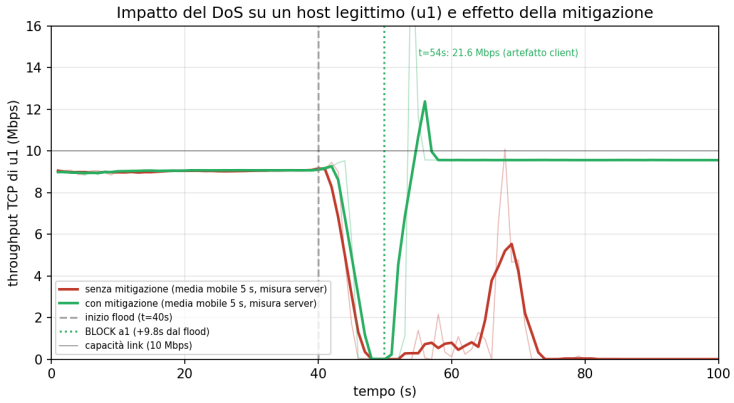


Figura 4: Throughput TCP di `u1`: tra +20 e +60 s dal flood scende a 2,3 Mbps senza mitigazione e resta a 9,6 Mbps con la mitigazione (9,0–9,1 Mbps prima del flood).

Come ci si è arrivati. Ogni problema è stato riprodotto con uno scenario Mininet, corretto e rimisurato: le regole di drop finivano nelle metriche (escluse); senza whitelist il TCP di `u1`, che satura il link, superava il tetto e veniva bloccato 23,5 s prima del flood (tetto solo per chi non riceve risposte); il flood a freddo passava sotto il tetto da 1,1 MB/s (abbassato a 0,95); `a2`

rallentava per un intervallo e ritardava il blocco (isteresi fino a 0,6 MB/s); sotto SYN flood le risposte del server sembravano un flood (chi riceve più di quanto manda non viene segnalato). Il rilevamento per congestione (`CONGESTION=1`) è un risultato negativo: il TCP di `u1` porta da solo l'uplink oltre il 90%, e senza whitelist veniva bloccato `u1`; resta disattivato.

Machine Learning. Dataset generato con `gen_dataset.sh`: 2098 righe, 100 episodi da 60 s con traffico benigno vario e attacchi hping3 (UDP, SYN, ICMP, low-rate). La prima valutazione dava 1,00, ma mescolava campioni dello stesso episodio tra training e test; ora si usa la **validazione per gruppo** (episodio e host) e le famiglie SYN e low-rate sono solo di test.

Metodo	Precision	Recall	F1	FPR	Attaccanti / onesti bloccati
Random Forest (profondità 4)	0,39	0,95	0,55	0,21	98% / 19%
Soglia statica (tetti)	0,33	0,57	0,42	0,17	60% / 18%
EWMA del baseline	0,00	0,00	0,00	0,01	0% / 2%
EWMA + tetti	0,32	0,65	0,43	0,20	71% / 21%

Tabella 3: Valutazione offline per campione e per host (conferma 2/3).

Il ML è tenuto come strategia alternativa: i falsi positivi vengono dall'UDP legittimo, che con le sole feature di flusso ha gli stessi pacchetti/s, byte/s e dimensione di un flood UDP; anche Gradient Boosting e MLP danno risultati simili, quindi il limite è nelle feature. Offline le soglie sono penalizzate, perché il dataset non registra il traffico di ritorno: in rete la stessa soglia non ha mai bloccato `u1`. Sulla famiglia SYN mai vista il Random Forest prende il 73% dei campioni.

5 Limiti e sviluppi futuri

- Uno streaming UDP legittimo ad alta banda non riceve risposte e supera il tetto: limitarne la banda con i *meter* OpenFlow invece di bloccarlo.
- Un flood verso un servizio che risponde aggira la regola del traffico di ritorno: feature di livello 4 (porte distinte, flag TCP) raccolte con il campionamento **sFlow** di OVS, fuori dal canale OpenFlow per non sovraccaricare il controller.
- MAC spoofing: blocco per porta d'ingresso e *port security* sullo switch di accesso.
- Circa 10 s per bloccare e ricadute ogni 60 s per un attaccante persistente: conferma immediata oltre il tetto e durata del blocco crescente.
- ML: disattivare l'offload TSO di Mininet, che falsa la dimensione dei pacchetti, e aggiungere feature L4.

6 Conclusioni

Il sistema blocca il flood in circa 10 s (12 s se parte a freddo), riporta l'utente legittimo da 2,3 a 9,6 Mbps e **non blocca mai il traffico onesto, anche senza whitelist**. L'idea centrale è semplice e verificabile: un flood manda tanto e non riceve niente. Codice, log e istruzioni di riproduzione sono nel repository `https://github.com/giovannistanco/NCIS_unina/tree/main/2026/Marrone_Bellotti`.